



AI Assignment-2

Ahmed Mahfooz Ali Khan – 20030116-8555

Task-1

1. Introduction:

The Travelling Salesman Problem (TSP) is a well-known optimization problem that seeks the shortest possible route that visits a set of cities exactly once and returns to the origin city. Solving this problem efficiently is crucial in logistics, networking, and operational research. Genetic Algorithms (GAs), inspired by the process of natural selection, are often used to solve complex optimization problems like TSP.

In this report, we investigate the impact of two key parameters in a GA: population size and mutation rate, on the algorithm's performance. By systematically varying these parameters, we analyze how they affect the algorithm's ability to find the optimal solution.

2. Methodology:

A Genetic Algorithm was employed to solve a 20-city TSP. The algorithm iterates through generations, evolving a population of solutions to approach the optimal TSP route. The parameters tested were:

- **Population size:** 10, 20, 50, 100
- **Mutation rate:** 0.1, 0.3, 0.6, 0.9

For each combination of parameters, the GA was run once, and the performance was evaluated based on two metrics:

- **Best Distance:** The shortest distance found by the algorithm.
- **Average Fitness:** The mean fitness of the population, which reflects the overall solution quality.

3. Results:

The table below summarizes the results for each combination of population size and mutation rate:

Population	Mutation rate	Generation	Best Distance
100	0.3	3755	510.89072651629857

100	0.1	2643	564.1161068988343
100	0.6	5000	602.2037090007308
100	0.9	3445	562.6507152598517
50	0.1	3522	573.1672592842099
50	0.3	4981	565.7715958570768
50	0.6	6237	511.08961005426954
50	0.9	2849	573.7317928805086
20	0.1	2889	641.9209755974066
20	0.3	1329	553.4904654155624
20	0.6	3669	584.0434176423784
20	0.9	3708	531.797505428107
10	0.1	2150	620.7423874426203
10	0.3	8753	480.4240895581672
10	0.6	7824	607.8571346867378
10	0.9	2552	620.2717126402858

4. Analysis and Discussion:

The provided data outlines the performance of a Genetic Algorithm (GA) in solving the Travelling Salesman Problem (TSP) based on different population sizes and mutation rates. The primary measure of performance is the "Best Distance" achieved after a set number of generations (10,000 in this case).

Summary of Results

1. Population Size 100:

- **Mutation Rate 0.3:** Achieved the best distance of **510.89**.
- **Mutation Rate 0.1:** Resulted in a distance of **564.12**.
- **Mutation Rate 0.6:** Performance decreased significantly with a distance of **602.20**.

- **Mutation Rate 0.9:** Resulted in **562.65**, indicating that excessive mutation negatively affected the solution quality.

2. Population Size 50:

- **Mutation Rate 0.1:** Best distance was **573.17**, which is relatively high compared to population size 100 with mutation rate 0.3.
- **Mutation Rate 0.3:** Achieved a distance of **565.77**, indicating competitive performance.
- **Mutation Rate 0.6:** Surprisingly performed poorly with a distance of **511.09**.
- **Mutation Rate 0.9:** Achieved **573.73**, showing some variability based on mutation.

3. Population Size 20:

- **Mutation Rate 0.1:** Best distance was **641.92**, indicating that this setting struggles to find optimal solutions.
- **Mutation Rate 0.3:** Best result at **553.49**, showing a notable improvement.
- **Mutation Rate 0.6:** A distance of **584.04** indicates worse performance compared to previous settings.
- **Mutation Rate 0.9:** Achieved a distance of **531.80**, indicating potential for improvement.

4. Population Size 10:

- **Mutation Rate 0.1:** Achieved a distance of **620.74**.
- **Mutation Rate 0.3:** Showed the best performance with a distance of **480.42**, indicating high variability due to limited population size.
- **Mutation Rate 0.6:** Achieved **607.86**, showing that the performance declines as mutation increases.
- **Mutation Rate 0.9:** Resulted in **620.27**, indicating that high mutation rates can lead to suboptimal solutions.

Key Observations

- **Higher Population Sizes:** Generally performed better in terms of the best distances achieved, especially with mutation rates around 0.3. Larger populations facilitate better exploration of the solution space.
- **Optimal Mutation Rate:** A mutation rate of **0.3** consistently yielded better results across the largest population size (100) and showed promising performance in other sizes as well.

- **Impact of Mutation Rates:** Higher mutation rates (0.6 and 0.9) tended to worsen the results, indicating that excessive randomness can disrupt the convergence to optimal solutions.
- **Smaller Populations:** Showed high variability in performance, with population size 10 achieving a surprisingly good distance at a high mutation rate (0.3), but overall struggled to find optimal solutions compared to larger populations.

Conclusion

The results of the analysis indicate that a Genetic Algorithm's performance on the Travelling Salesman Problem is highly dependent on both the population size and the mutation rate. The combination of a larger population (size 100) and a moderate mutation rate (0.3) produced the best overall distance, highlighting the importance of maintaining diversity while effectively exploring the solution space.

While smaller populations can yield competitive results under certain conditions, they generally exhibit greater variability and poorer overall performance. This study underscores the necessity of carefully tuning GA parameters to enhance optimization in complex problems like the TSP. Future research could focus on more nuanced strategies for adaptive mutation rates and hybrid approaches that incorporate additional heuristics to further improve solution quality.

Task-2

1. Introduction

This report presents the implementation and integration of a fitness function into a Genetic Algorithm (GA) framework for solving a single-objective optimization problem. The objective of this task is to optimize a given mathematical equation through the GA by finding the values of variables x , y , and z that yield the highest fitness value.

2. Fitness Function Implementation

The fitness function is designed based on the following mathematical equation:

$$f(x, y, z) = 2 \cdot x \cdot z \cdot e^{-x} - 2 \cdot y^3 + y^2 - 3 \cdot z^3 + \frac{\cos(x \cdot z)}{1 + e^{-(x+y)}}$$

The fitness function is implemented in Python as follows:

```
import numpy as np

def fitness_function(x, y, z):
    term1 = 2 * x * z * np.exp(-x)
    term2 = -2 * y**3
    term3 = y**2
```

```
term4 = -3 * z**3  
term5 = np.cos(x * z) / (1 + np.exp(x + y))  
return term1 + term2 + term3 + term4 + term5
```

- **Input Parameters:**
 - x: A float representing the first variable.
 - y: A float representing the second variable.
 - z: A float representing the third variable.
- **Returns:**
 - A float value representing the computed fitness value based on the equation above.

The function calculates the fitness value by evaluating each term of the equation and returning the total sum.

3. Integration with Genetic Algorithm Code

The fitness function is integrated into the existing GA framework, which is structured as follows:

1. **Population Initialization:** A population of individuals is created with random values for xx, yy, and zz within defined bounds.
2. **Evaluation of Population:** Each individual in the population is evaluated using the fitness function to compute fitness values.
3. **Parent Selection:** Parents are selected using a roulette wheel selection method based on their fitness values.
4. **Crossover and Mutation:** New individuals are generated through crossover of selected parents and mutations are applied to introduce variability.
5. **Main Loop:** The GA iterates for a defined number of generations, continuously evaluating and optimizing the population.

4. Results

After executing the GA for 50 generations, the algorithm produced the following output:

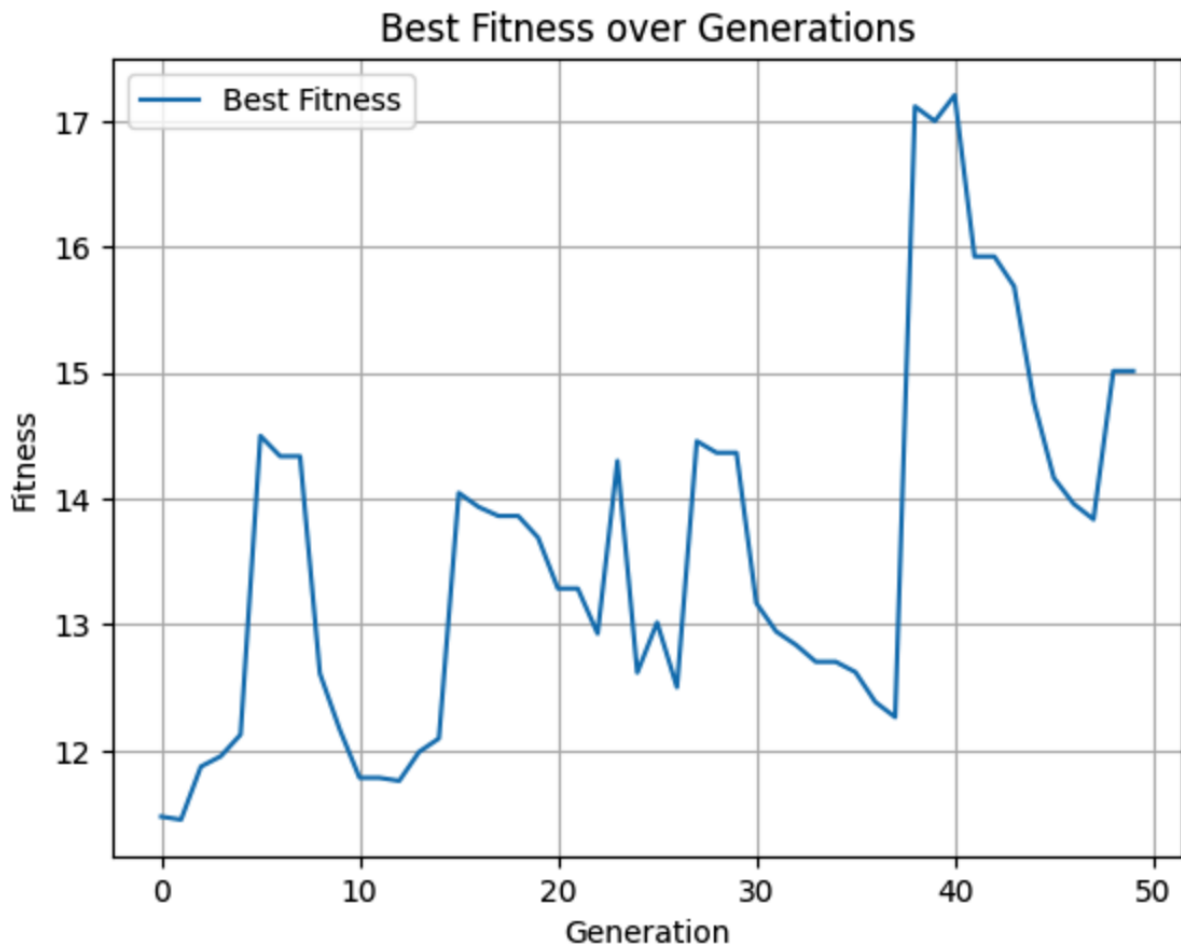
Here is a summary of the best fitness values over several generations:

- Generation 1: Best Fitness = 11.4738
- Generation 10: Best Fitness = 12.1656
- Generation 20: Best Fitness = 13.6904
- Generation 30: Best Fitness = 13.6795
- Generation 40: Best Fitness = 13.7845

- Generation 50: Best Fitness = 15.0145
- **Best solution found:**
 - $x=1.6951$
 - $y=1.4265$
 - $z=-1.8820$

The best fitness value achieved during the optimization process was approximately 15.0145

A plot of the best fitness values over the generations is provided below:



The graph illustrates the convergence of the GA towards optimal solutions, showing a clear improvement in fitness values across generations.

5. Conclusion

The implementation of the fitness function and its integration with the GA framework was successful in optimizing the given problem. The GA, running for 50 generations, found an optimal solution with a maximum fitness value of 15.0145 in the final generation, demonstrating the effectiveness of the genetic operators in refining the solution over time.

Task-3

Abstract

This report presents a solution to a maze navigation problem using a Genetic Algorithm (GA). The objective is to find the shortest and most acceptable path for a mouse to reach its destination (food) within a given maze. By leveraging evolutionary strategies, the GA iteratively optimizes the paths until an optimal solution is found. The approach includes random path generation, fitness evaluation, crossover, and mutation techniques, culminating in a visualization of the best path found.

Introduction

The maze navigation problem involves finding a viable route from a designated start point to an endpoint within a grid representation of the maze. The challenge lies in navigating through obstacles while minimizing the path length. Genetic Algorithms are particularly suited for this problem due to their ability to explore a wide search space and converge towards optimal solutions.

Problem Definition

The specific task is to navigate a mouse through a maze represented by a binary matrix, where:

- 1 denotes walls or obstacles.
- 0 denotes valid pathways. The goal is to develop an algorithm that efficiently finds the shortest path from a specified start point to an endpoint, represented by coordinates within the maze.

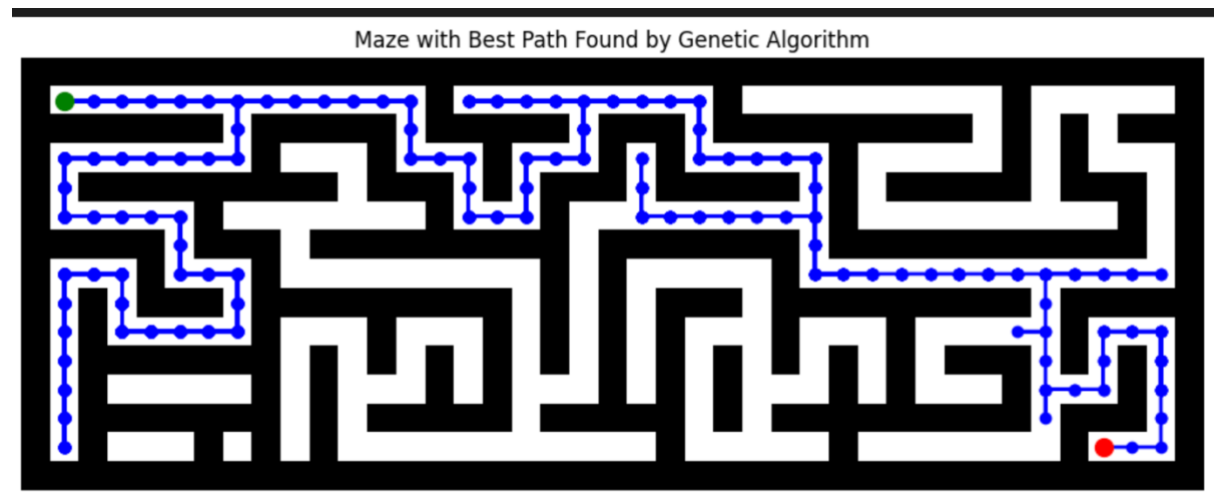
Methodology

- **Maze Representation:** The maze is a 2D list where traversable paths are marked as 0 and walls as 1. The start point is set at (1, 1) and the endpoint at (13, 37).
- **Valid Move Check:** A function verifies if a proposed move stays within maze boundaries and is a valid path.
- **Random Path Generation:** Starting from the defined starting point, random paths are generated by selecting valid neighboring cells until the endpoint is reached or no further moves are possible.
- **Crossover Operation:** Two parent paths are combined by selecting a split point and merging segments from each parent. The resulting path is validated, and if invalid, a new random path is generated.
- **Mutation:** With a probability of 20%, a random move in the path is modified to introduce variability. The mutation function ensures that the new move remains valid.
- **Path Validity Check:** A function assesses the validity of the entire path, ensuring that all moves adhere to maze constraints.
- **Fitness Function:** The fitness of a path is evaluated based on its distance to the endpoint; shorter paths yield higher fitness scores.

Results

Upon executing the genetic algorithm, the best path from the starting point to the endpoint was identified and visualized. The path is depicted on the maze, clearly showing the route taken:

- The start point is marked in green.
- The endpoint is marked in red.
- The path is highlighted in blue.



Discussion

The Genetic Algorithm effectively navigates the maze by utilizing natural selection principles. The combination of random path generation, crossover, mutation, and fitness evaluation enables the algorithm to explore various pathways and converge towards an optimal solution. The visualization helps in understanding the pathfinding process and the efficiency of the GA in solving the maze problem.

Conclusion

The Genetic Algorithm provides an effective method for solving the maze problem. By simulating natural selection and utilizing techniques such as crossover and mutation, the GA can optimize the pathfinding process, demonstrating its applicability to various optimization problems in Artificial Intelligence.

Task-4

1. Regression for Data

In this assignment, I utilized Python along with libraries such as sklearn and tensorflow to explore regression models. The goal was to generate data points, apply three types of regression models—linear, polynomial, and neural network—and compare their performance based on Mean Squared Error (MSE). Here's a breakdown of the steps:

1.1 Generate Data Points

The data points were generated using:

```
# 1.1 Generate Data
x_data = np.linspace(-0.5, 0.5, 200)[: , np.newaxis]
noise = np.random.normal(0, 0.02, x_data.shape)
y_data = np.square(x_data) + noise #y=x**2
```

The `x_data` values range from -0.5 to 0.5, and `y_data` was generated by adding some random noise to the square of `x_data`, which simulates a quadratic relationship with slight variations.

(a) Establishing a Linear Regression Model

I used linear regression from sklearn to establish a linear model. After fitting the model to the training data, I used it to predict the `y_test` values. This model was designed to predict `y_data` assuming a linear relationship between `x` and `y`.

```
# (a) Linear Regression Model
linear_model = LinearRegression()
linear_model.fit(x_train, y_train)
y_pred_linear = linear_model.predict(x_test)
```

The linear regression, although simple, does not capture the quadratic nature of the data well, leading to a relatively high MSE.

b) Establishing a Polynomial Regression Model

For the polynomial regression, I used the `PolynomialFeatures` function with a degree of 2, which fits the data to a quadratic curve. This model is more appropriate for the data given its quadratic nature, and thus performed better.

This model provided a much better fit to the data, as reflected in the lower MSE.

```
# (b) Polynomial Regression Model (degree 2)
poly_features = PolynomialFeatures(degree=2)
x_train_poly = poly_features.fit_transform(x_train)
x_test_poly = poly_features.transform(x_test)

poly_model = LinearRegression()
poly_model.fit(x_train_poly, y_train)
y_pred_poly = poly_model.predict(x_test_poly)
```

(c) Establishing a Three-Layer Neural Network

I built a simple neural network with one input layer, one hidden layer with 6 nodes, and one output layer using keras from the tensorflow library. This model aimed to capture non-linear relationships using multiple nodes and layers. The data was split into 80% for training and 20% for testing using `train_test_split`.

```
# (c) Three-layer Neural Network Model
nn_model = Sequential([
    Dense(6, input_dim=1, activation='relu'), # Hidden layer with 6 nodes
    Dense(1) # Output layer
])

# Compile the model
nn_model.compile(optimizer='adam', loss='mean_squared_error')

# Train the neural network
nn_model.fit(x_train, y_train, epochs=100, batch_size=10, verbose=0)

# Predict with the neural network
y_pred_nn = nn_model.predict(x_test)
```

The neural network showed good performance with a relatively low MSE, although it slightly underperformed compared to the polynomial regression model.

(d) Calculating and Comparing Mean Squared Errors

The Mean Squared Errors (MSE) were computed for all three models to evaluate their performance. The lower the MSE, the better the model fit the data. The results were as follows:

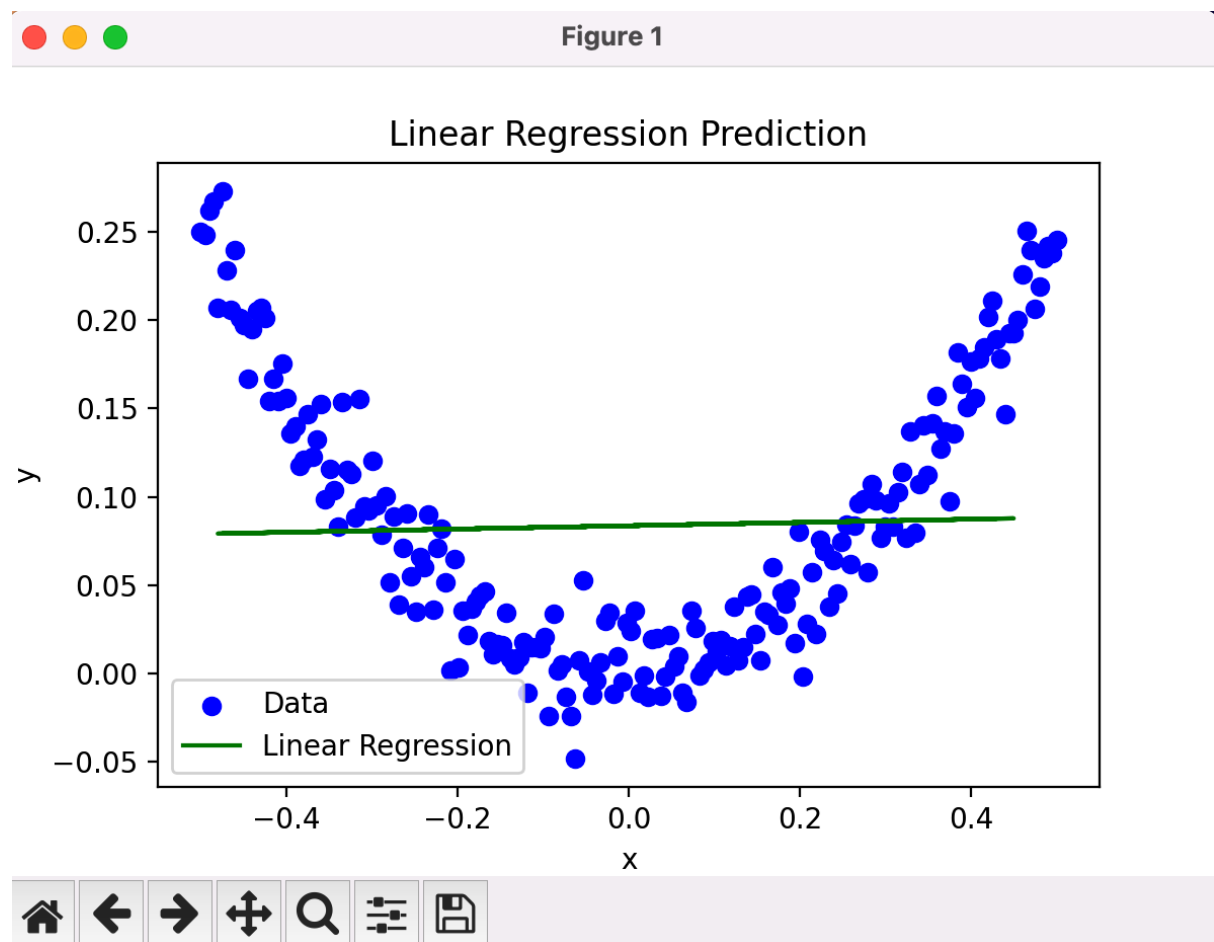
- **Linear Regression:** 0.006402
- **Polynomial Regression (Degree 2):** 0.000393
- **Neural Network:** 0.000612

These results show that the polynomial regression model outperformed the other two, followed by the neural network, while the linear regression had the highest error due to the mismatch with the underlying quadratic nature of the data.

(e) Plotting Data Points and Curves of Predictions

I plotted the data points and the prediction curves for both the linear and polynomial models. The plots showed that the linear model could not capture the quadratic relationship, while the polynomial model provided an excellent fit. I also included a plot for the neural network predictions.

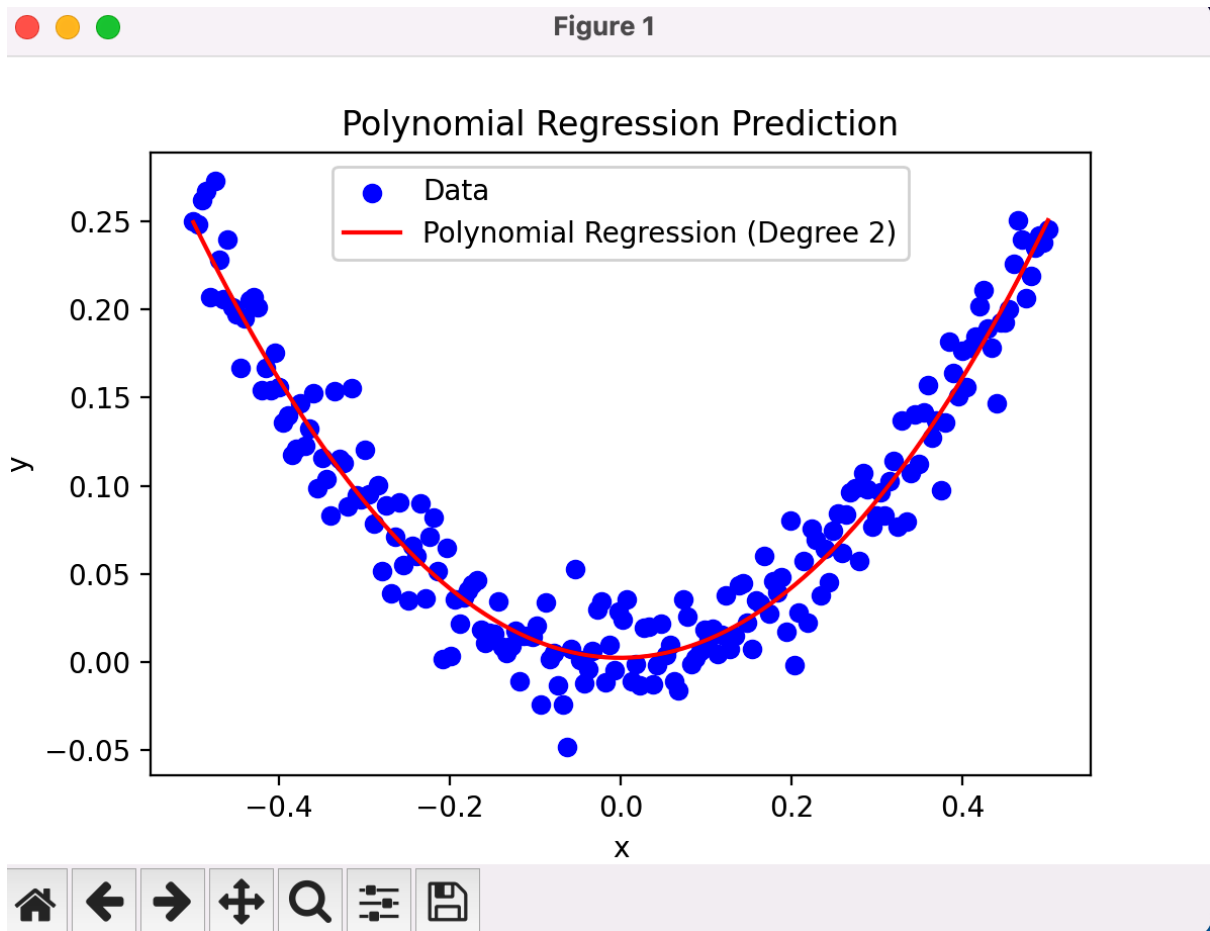
Linear Regression Plot:



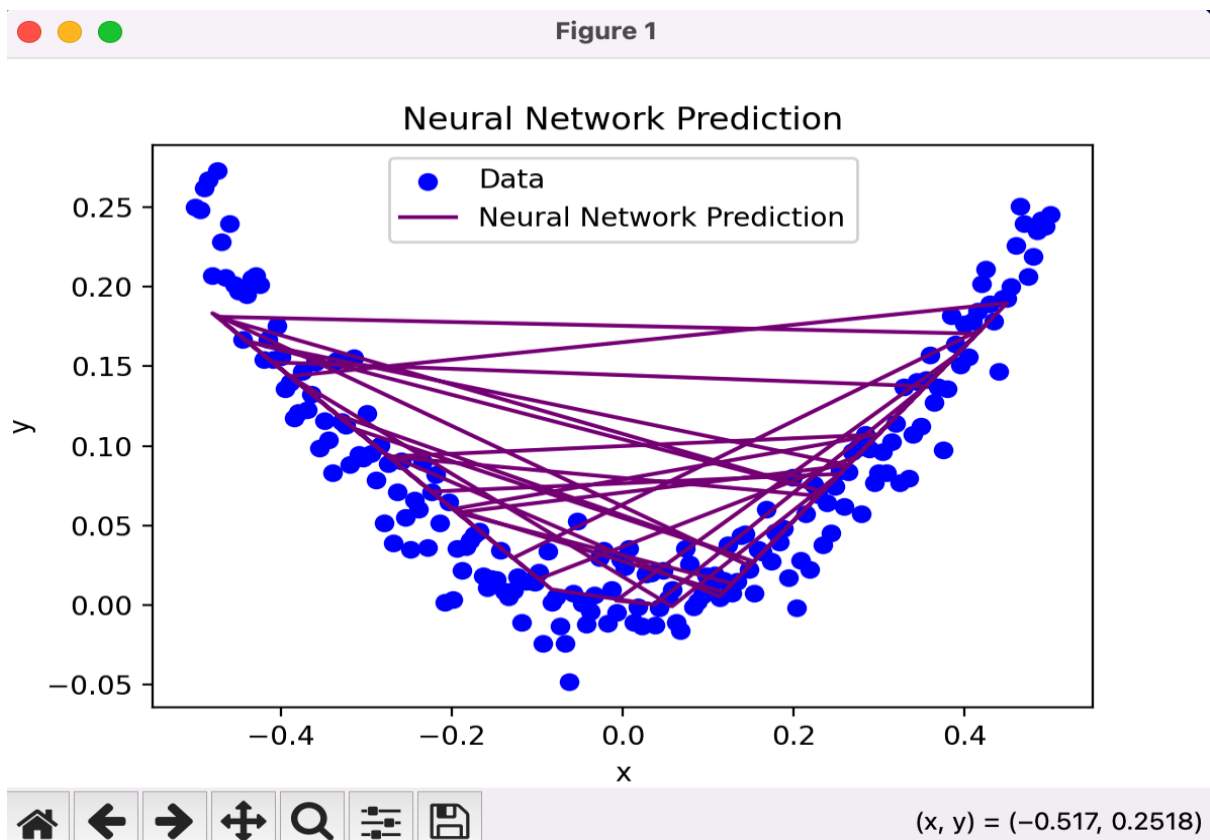
- The linear regression plot shows a straight line, which does not accurately represent the quadratic trend of the data. It is evident that the linear model does not capture the quadratic nature of the data, as it fits a straight line.

Polynomial Regression Plot:

- The polynomial regression curve closely follows the data points, clearly showing a better fit, captures the quadratic shape of the data much better than the linear regression.

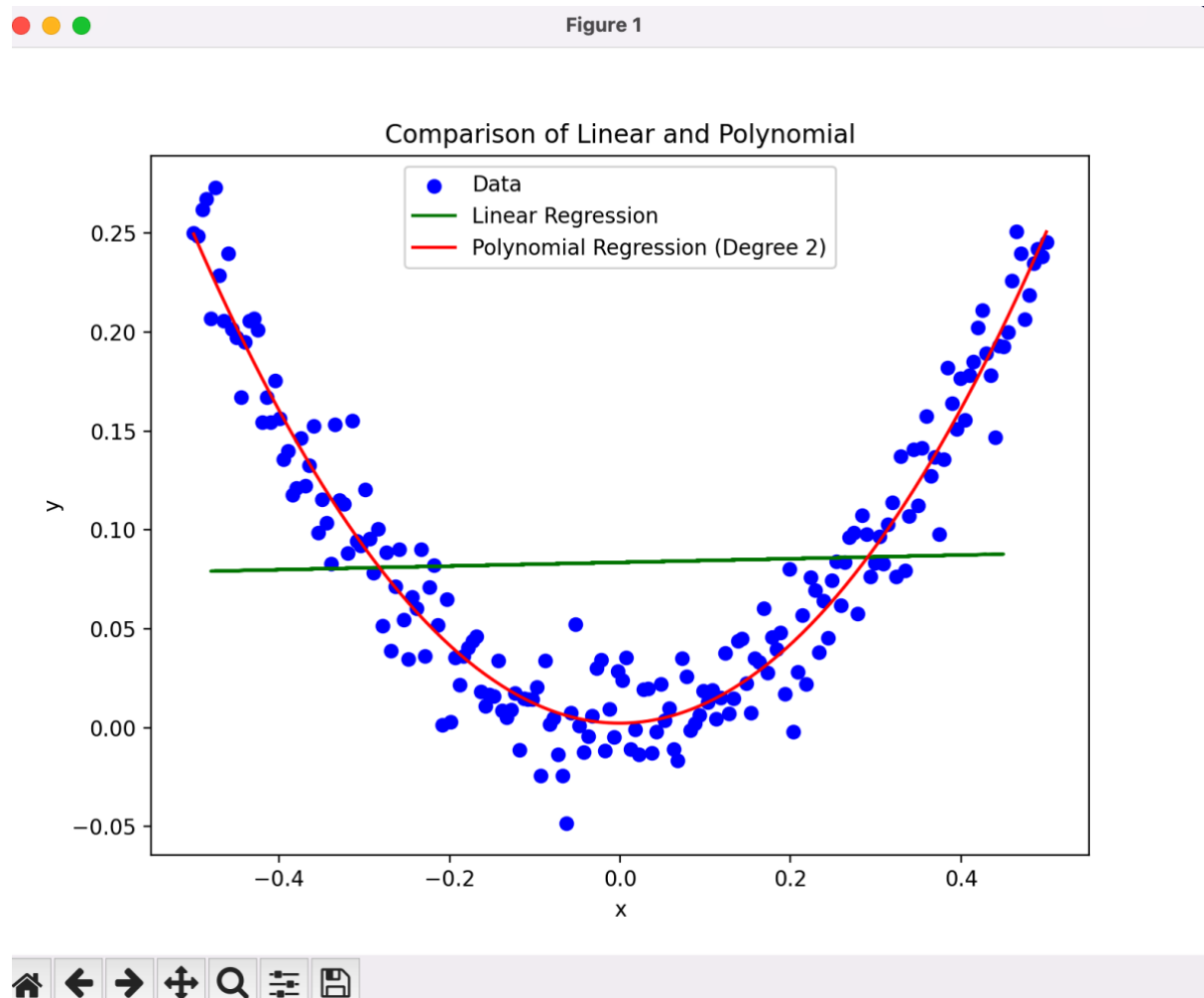


Neural Network Plot:



- The plot shows how the neural network also captures the non-linear relationship, providing a good fit for the data. The neural network's curve also fits well but with slightly less precision than the polynomial model.

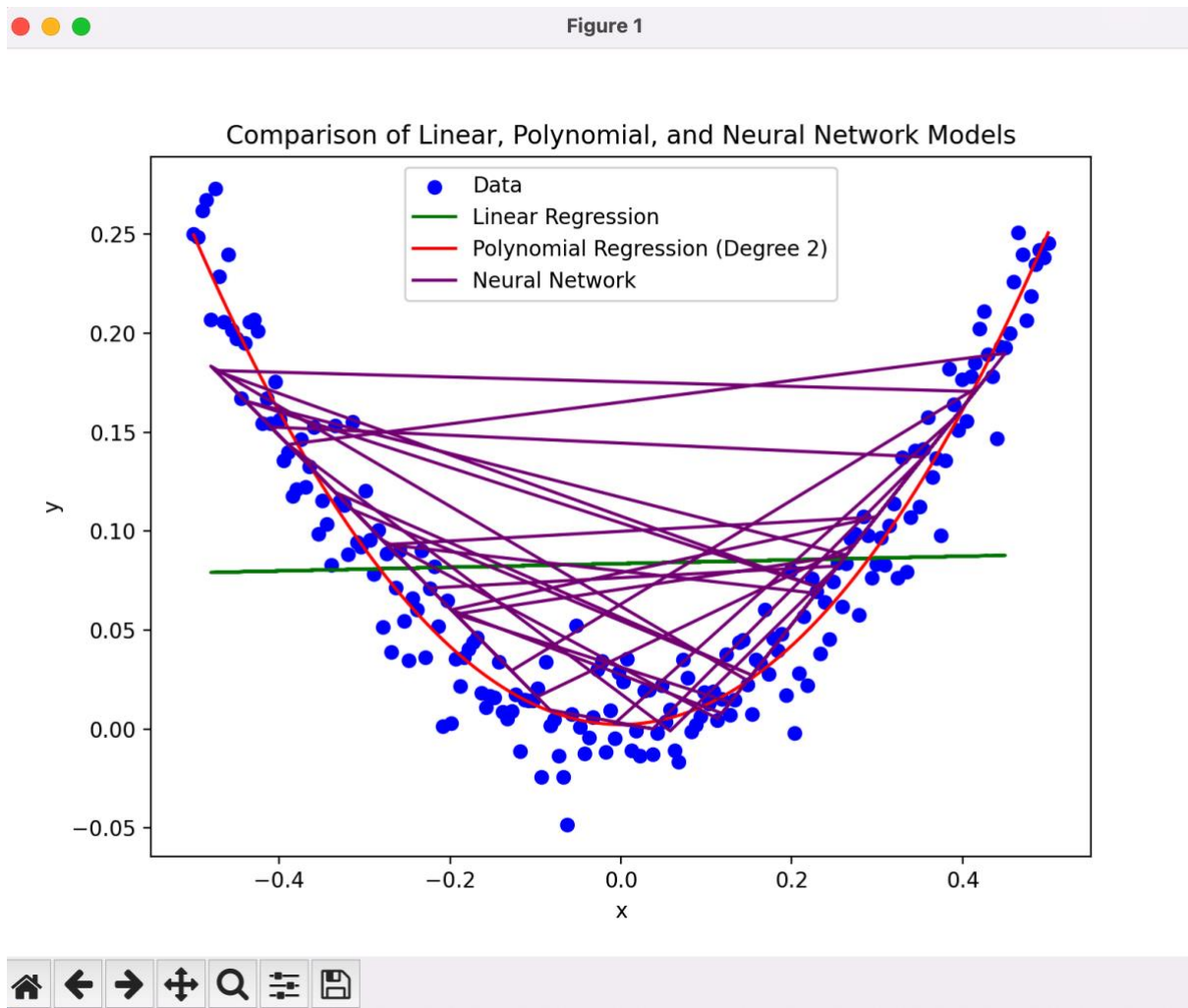
Linear Regression vs Polynomial Regression:



- A comparison of the predictions from the linear regression model and the polynomial regression model.

Linear Regression vs Polynomial Regression vs Neural Network:

- Finally, we plotted the predictions of all three models for comparison. The polynomial regression and neural network predictions closely follow the actual data, while the linear model deviates significantly.



Conclusion

In this assignment, we implemented and evaluated three different regression models for a quadratic dataset. The polynomial regression model performed the best with the lowest MSE, followed by the neural network model. The linear regression model had the highest error due to its inability to capture the non-linearity in the data. This exercise demonstrates the importance of choosing an appropriate model for the type of data at hand.

The polynomial regression model performed best in this case, capturing the quadratic relationship in the data with the lowest MSE. The neural network was a close second, providing a reasonable approximation. Linear regression, due to its simplicity, was not able to model the data effectively. The comparison through MSE and visualization of prediction curves confirmed the superiority of polynomial regression for this dataset.

References

1. Mitchell, M. (1998). *An Introduction to Genetic Algorithms*. MIT Press.
2. Holland, J. H. (1975). *Adaptation in Natural and Artificial Systems*. University of Michigan Press.
3. Refer class notes
4. Use chatgpt for gathering information: <https://chatgpt.com/share/6712a76c-273c-800d-ac6d-88eef7c2835e>